Ledbetter Data Pump (LDP)

Addendum: Failure Modes, Edge Cases & Recovery Semantics

Companion to the Architecture White Paper — Buildable Solutions Included

Version 1.0 • November 2026

Ledbetter Data Pump (LDP)

Addendum: Failure Modes, Edge Cases & Recovery Semantics

Version 1.0

Date: November 2026

Status: Companion to Architecture White Paper — Buildable Solutions Included

This addendum does **not** replace the core architecture white paper. It extends it with explicit handling of non-happy-path behavior. Every item below states:

1. The problem / edge case
2. The required behavior
3. A concrete, buildable architectural solution
4. The phase in which it should be implemented

All solutions are designed to fit the existing control-plane + Rust engine + Arrow RecordBatch model.

1. Expression Evaluation Failures

Problem

A single bad value (null, type error, overflow, division by zero, invalid date, etc.) can crash a worker or the entire job.

Required Behavior

Bad rows are isolated. The job continues or fails according to the configured error policy. Good rows are never lost.

Architectural Solution (Buildable)

• Evaluate expressions with Arrow compute kernels (or a thin DataFusion physical expression layer).

• Wrap every transform stage in a row-level try/catch boundary.

• On failure:

- Increment error counter

- Emit the original row + error message to the Dead Letter Queue (DLQ)

- Drop or null the bad row according to policy

• Support two modes in YAML:

- `fail_fast`

- `dlq_and_continue` (with `max_bad_rows` / `max_error_percentage`)

• Expression language for v1 is deliberately small and safe: comparisons, arithmetic, `IS NULL`, `IN`, simple functions (`now()`, `coalesce`). No arbitrary code.

Phase: 1 (basic) → 2 (full DLQ + budgets)

2. Schema Drift and Type Mismatches

Problem

Source schema changes (added/dropped/renamed columns, decimal scale change, timezone differences, JSON vs string) cause silent corruption or crashes.

Required Behavior

The engine either fails fast with a clear message or adapts according to an explicit policy. No silent data loss.

Architectural Solution (Buildable)

• Require an optional `schema` section in the source definition (or auto-infer + lock on first run).

• Central `TypeMapper` module that owns all Arrow ↔ SQL type conversions.

• YAML `schema_policy`:

- `strict` — fail if source schema differs from expected

- `lenient` — extra columns ignored, missing columns become null, safe coercions applied, unsafe coercions sent to DLQ

• Log every coercion and schema difference at INFO/WARN.

• Persist the observed schema in the run state for audit.

Phase: 2

3. Partition Skew and Empty Partitions

Problem

One partition contains almost all the data; others are empty. Workers finish early or one worker OOMs / times out.

Required Behavior

The engine detects and reports skew. Empty partitions are cheap. Operators can override partitioning.

Architectural Solution (Buildable)

• After planning partitions, optionally run a lightweight count or sample query per partition (configurable).

• Emit partition statistics (estimated rows) before execution.

• Support `partitioning.type: none` or `chunks: 1` as an explicit single-partition fallback.

• Allow manual `partition_ranges` override in YAML for known skewed tables.

• Log a warning when max/min partition size ratio exceeds a threshold (e.g., 10×).

Phase: 1 (basic fallback) → 2 (stats + warnings)

4. Multi-Target Partial Failure

Problem

One sink succeeds, another fails. Restarting can duplicate data or leave inconsistent state.

Required Behavior

Clear, configurable policy. Default is all-or-nothing for the chunk.

Architectural Solution (Buildable)

• A chunk is marked complete in the checkpoint **only after all sinks for that chunk succeed**.

• YAML `multi_target_policy`:

- `all_or_nothing` (default)

- `best_effort` (continue, report partial success in final summary)

• Staging tables / temp files are used so a failed sink can be retried without duplicating already-written data.

• Final run summary always lists per-target status.

Phase: 2

5. Memory Pressure and Oversized Rows

Problem

Wide rows, large JSON/text/binary columns, or expanding transforms cause OOM even with bounded channels.

Required Behavior

Hard limits prevent unbounded memory growth. Oversized rows are isolated or rejected.

Architectural Solution (Buildable)

• Global and per-worker memory budgets (configurable in settings).

• Hard limits:

- `max_batch_rows`

- `max_batch_bytes`

- `max_row_bytes`

• If a single row exceeds `max_row_bytes` → send to DLQ and continue (or fail according to policy).

• Arrow RecordBatch builder enforces batch size limits before emission.

• Optional spill-to-disk for intermediate batches (Phase 3+).

Phase: 1 (basic limits) → 2 (row-level isolation)

6. Source Connection / Query Failures Mid-Partition

Problem

Network blip, timeout, or lock causes a partition read to fail halfway.

Required Behavior

Retry the partition. Only mark it failed after retries are exhausted. Already-completed partitions stay completed.

Architectural Solution (Buildable)

• Partition-level retry with exponential backoff (configurable attempts).

• Reader is idempotent: re-executing the same partition range produces the same result.

• Checkpoint records only fully successful partitions.

• Transient errors (connection reset, timeout) are retried; permanent errors (syntax, permission) fail immediately.

Phase: 1

7. Target Constraint Violations

Problem

Primary key, unique, check, or foreign-key violations on the sink abort a batch or leave partial data.

Required Behavior

Configurable isolation. Prefer failing at staging time rather than after production write.

Architectural Solution (Buildable)

• Prefer atomic staging pattern: write to staging table, then swap/merge. Constraints are validated during the final swap when possible.

• For direct append/merge:

- Configurable `on_constraint_violation`: `fail_batch` | `dlq_batch` | `dlq_row` (where driver supports it)

• Batch-level DLQ captures the entire failed batch with the database error.

Phase: 2

8. Checkpoint Integrity and Concurrent Runs

Problem

Crash while writing state file, corrupted state, or two jobs running against the same pipeline.

Required Behavior

State writes are atomic. Concurrent runs are detected and rejected (or queued).

Architectural Solution (Buildable)

• Atomic state update: write to `state.json.tmp` then `rename` (atomic on same filesystem).
• Optional SQLite WAL backend for stronger durability.
• Run lock file or database advisory lock containing `run_id` + PID + start time.
• On startup, if lock exists and process is alive → refuse to start.
• State file contains schema version for future migration.

Phase: 1 (atomic write) → 2 (locking + SQLite option)

9. Watermark Edge Cases

Problem

Nulls in watermark column, clock skew, late data, or non-monotonic values cause missed or duplicated rows.

Required Behavior

Documented, predictable watermark semantics. Operators can see the values used.

Architectural Solution (Buildable)

• Watermark = `MAX(non-null values)` of the column for the successful run.
• Nulls in the watermark column are ignored for the max calculation (logged).
• Support optional `lookback` interval (e.g., `lookback: 1h`) to re-process recent data.
• Persist both previous and new watermark in state.
• Log the watermark value used in the query at INFO.

Phase: 2

10. File I/O Failures and Partial Writes

Problem

Disk full, permission errors, process kill during Parquet/CSV write produces corrupt output.

Required Behavior

Output files are either complete or absent. No partial visible files.

Architectural Solution (Buildable)

• Write all file sinks to a temporary path (`*.tmp` or `part-XXXX.tmp`).

• On success: atomic rename into final location.

• On failure: delete temp files.

• Parquet writer uses Arrow's streaming writer with proper footer finalization only on close.

• Encoding errors in CSV → row to DLQ.

Phase: 1

11. Empty Results and Zero-Row Partitions

Problem

Source returns zero rows or all rows are filtered. Downstream systems or operators assume failure or hang.

Required Behavior

Empty result is a successful outcome. Checkpoint is still written. Clear metrics are emitted.

Architectural Solution (Buildable)

• Zero-row RecordBatch is valid and flows through the pipeline.

• Sinks receive an empty batch and finalize cleanly (create empty Parquet, or no-op for SQL).

• Final summary reports `rows_read: 0`, `rows_written: 0`, `status: success`.

• Empty partitions are marked complete immediately.

Phase: 1

12. Graceful Shutdown and Cancellation

Problem

SIGTERM or user cancel leaves half-written staging data and unclear checkpoint state.

Required Behavior

Completed chunks are preserved. In-flight chunks are aborted cleanly. Job can be resumed.

Architectural Solution (Buildable)

• Install signal handler (tokio or std).

• On shutdown signal:

- Stop accepting new partitions

- Allow in-flight batches to finish or abort after a short timeout

- Flush checkpoint of fully completed chunks only

- Clean up temporary staging resources for incomplete chunks

• Final log line indicates `status: cancelled` and last completed chunk.

Phase: 2

13. Semantic Validation of YAML

Problem

YAML passes schema validation but is semantically wrong (partition column not in SELECT, circular stage references, missing input, etc.).

Required Behavior

Fail at planning time with a clear error before any data movement.

Architectural Solution (Buildable)

• After schema validation, run a semantic checker:

- All `input` references exist

- No cycles in the DAG

- Partition column appears in the source query (best-effort parse or user assertion)

- Target primary key columns exist in the incoming schema

• Emit all errors at once (not fail-fast on first).

Phase: 1

Summary of Phasing

Conclusion

These solutions are intentionally concrete and fit the existing architecture. They turn the most common production failure modes into designed, testable behavior rather than surprises.

Implement Phase 1 solutions together with the core engine. Treat the Phase 2 items as required before calling the system production-ready for unattended runs.

This addendum + the original white paper together constitute the full build contract.

End of Addendum